

Code – Lab 6

```
<!DOCTYPE html>

<html>

<head>

<title>Lab 6 - Hoisting Demo</title>

<style>

*{margin:0;padding:0;box-sizing:border-box}

body{font-family:Arial;background:linear-gradient(135deg,#f4e4d7,#faf0e6);padding:2rem}

.h{background:linear-
gradient(135deg,#8B4513,#D2691E);color:white;padding:2rem;border-
radius:12px;margin-bottom:2rem;text-align:center}

.s{background:white;padding:1.5rem;margin-bottom:1.5rem;border-radius:12px;box-
shadow:0 5px 20px rgba(0,0,0,.1)}

.s h2{color:#8B4513;margin-bottom:1rem;border-bottom:3px solid #D2691E;padding-
bottom:.5rem}

.cb{background:#f5f5f5;border:2px solid #ddd;padding:1rem;margin:1rem 0;border-
radius:6px;font-family:'Courier New',monospace;font-size:.9rem;overflow-x:auto}

.ob{background:#e7f3ff;border-left:4px solid #2196F3;padding:1rem;margin:1rem
0;border-radius:6px}

button{background:linear-
gradient(135deg,#D2691E,#8B4513);color:white;border:none;padding:.7rem
1.5rem;border-radius:6px;cursor:pointer;font-weight:bold;margin:.5rem}

button:hover{transform:translateY(-2px);box-shadow:0 5px 15px rgba(210,105,30,.3)}

.res{padding:1rem;background:#fff9e6;border:2px solid #FFD700;border-
radius:6px;margin:1rem 0}

</style>

</head>

<body>
```



```
<div class="s"><h2>6. Complete Demo</h2><button onclick="dAll()">Run  
Complete</button><div id="rAll" class="res" style="display:none"></div></div>  
  
</div>
```

```
<script>
```

```
function d1(){const r=['✓ Function called BEFORE declaration','Result:  
' +getOrderPrice(5)];document.getElementById('r1').innerHTML=r.join('<br>');document.get  
ElementById('r1').style.display='block'}  
  
function getOrderPrice(q){return q*4.50}
```

```
function d2(){const r=['var coffeeType hoisted to undefined','Before: '+coffeeType];var  
coffeeType='Espresso';r.push('After:  
' +coffeeType);document.getElementById('r2').innerHTML=r.join('<br>');document.getElem  
entById('r2').style.display='block'}
```

```
function d3(){const r=['let & const in Temporal Dead Zone','Accessing before declaration =  
ReferenceError','After declaration:  
accessible'];document.getElementById('r3').innerHTML=r.join('<br>');document.getEleme  
ntById('r3').style.display='block'}
```

```
function d4(){const r=['var has function scope, let has block scope'];try{function  
t(){if(true){var fv=3.50;let bv='Small'}r.push('var funcVar accessible: '+fv);try{r.push('let  
blockVar: '+bv)}catch(e){r.push('let blockVar: ReferenceError (block  
scoped)')}}t()}catch(e){}document.getElementById('r4').innerHTML=r.join('<br>');document.  
getElementById('r4').style.display='block'}
```

```
function d5(){const r=['Function expressions are NOT hoisted','var calcTotal = undefined  
(before)'];var calcTotal=function(a,b){return a*b};r.push('After assignment:  
' +calcTotal(4.50,2));document.getElementById('r5').innerHTML=r.join('<br>');document.get  
ElementById('r5').style.display='block'}
```

```
function dAll(){const r=['=== Pop-N-Dine Hoisting ===',1. Function called before:  
'+getCoffeePrice('Espresso'),'2. var menuOpen: true'];var menuOpen=true;r.push('3. Block  
scope test');{let cp=4.50;var df=2.00;r.push('Inside: let='+cp+', var='+df)}r.push('Outside:  
var='+df);document.getElementById('rAll').innerHTML=r.join('<br>');document.getElement  
ById('rAll').style.display='block'}
```

```
function getCoffeePrice(t){const p={'Espresso':3.50,'Cappuccino':4.50,'Latte':4.50};return  
p[t]||0}
```

```
</script>
```

```
</body>
```

```
</html>
```

Output –

 **Function & Variable Hoisting**
Understanding JS Hoisting & Scope

1. Function Hoisting

Function declarations are hoisted - accessible before declaration

```
console.log(getOrderPrice(5)); // Works!
```

Run Demo 1

2. Variable Hoisting (var)

var is hoisted but initialized to undefined

```
console.log(coffeeType); // undefined  
var coffeeType = "Espresso"; // Now assigned
```

Run Demo 2

3. let & const (Temporal Dead Zone)

let/const are hoisted but NOT initialized - ReferenceError before

```
// console.log(orderSize); // ReferenceError!  
let orderSize = "Large";
```

Run Demo 3

4. Scope: var vs let

var has function scope, let has block scope

```
if(true) {  
  var funcVar = 3.50; // Function scope  
  let blockVar = "Small"; // Block scope  
}  
console.log(funcVar); // Works  
console.log(blockVar); // ReferenceError
```

Run Demo 4

5. Function Expression

Function expressions are NOT hoisted (treated as var)

```
// calcTotal(); // TypeError: not a function  
var calcTotal = function(x,y){ return x*y; };
```

Run Demo 5

6. Complete Demo

Run Complete

